UNIVERSIDADE DA CORUÑA

PRÁCTICA 1

Smart Contract de Reventa Antifraude

TECNOLOGÍAS DE REGISTRO DISTRIBUIDO
Y BLOCKCHAIN (BC)

Diego Rivero de la Hoz
Mauro Zelenka Pedrosa

MÁSTER INTERUNIVERSITARIO EN
CIBERSEGURIDAD

Curso 2026 – 2027

Índice General

1. Análisis y Definición del Escenario **3**

1.1. El Problema del Billetaje Tradicional y los Mercados de Reventa 3

1.2. Justificación de una Blockchain Pública (Ethereum) 3

2. Diseño del Sistema **4**

2.1. Lógica de Negocio y Ciclo de Vida del Ticket 4

2.2. Contenido del Contrato: Datos, Funciones y Reglas de Operación 4

2.3. Actores del Sistema y Diagrama de Casos de Uso 5

3. Implementación y Optimización de Gas **7**

3.1. Código del Smart Contract (`TicketSystemBasic.sol`) 7

4. Pruebas y Validación en Remix IDE **11**

4.1. Entorno de Pruebas y Cuentas Asignadas 11

4.2. Batería de Pruebas Ejecutadas Paso a Paso 11

Análisis y Definición del Escenario

Comprar una entrada para un concierto o un evento deportivo se ha convertido en un problema para los usuarios: las taquillas se agotan en segundos por programas automáticos, se revenden a precios desorbitados y abundan los engaños con entradas falsas o duplicadas. Frente a estos abusos, la tecnología blockchain permite crear un sistema con reglas automáticas e inviolables que protegen al comprador y acaban con la especulación.

1.1 El Problema del Billetaje Tradicional y los Mercados de Reventa

Los sistemas actuales de venta electrónica presentan tres fallos principales:

1. **Entradas duplicadas y falsificaciones:** Un archivo PDF o un código QR se puede copiar, reenviar por mensajería o fotografiar tantas veces como se quiera. En la puerta de acceso no es posible distinguir cuál es la copia original y cuáles son duplicados.
2. **Reventa abusiva y acaparamiento:** Redes organizadas compran lotes de entradas al instante usando bots automáticos y las revenden multiplicando su precio, sin que este sobrecoste beneficie a los organizadores ni a los artistas.
3. **Estafas entre particulares:** En plataformas de segunda mano, el comprador paga a ciegas mediante transferencia o Bizum sin ninguna garantía de recibir la entrada o de que el vendedor no se la haya vendido a varias personas a la vez.

1.2 Justificación de una Blockchain Pública (Ethereum)

La red Ethereum resuelve estos problemas sustituyendo la confianza en intermediarios por reglas que el Smart Contract ejecuta de manera automática:

- **Cada entrada es única e infalsificable:** Cada localidad pertenece a la cuenta digital de su comprador. No se puede duplicar ni fotocopiar; solo su dueño legítimo puede transferirla o ponerla en reventa.
- **Límite de precio obligatorio:** El contrato fija un recargo máximo para la reventa (por ejemplo, un $+10 \setminus \%$ sobre el precio original). El sistema rechaza automáticamente cualquier intento de venta por encima de ese límite.
- **Intercambio seguro y al instante:** El pago del dinero y la entrega de la entrada ocurren a la vez en una única operación atómica. El vendedor recibe el dinero y el comprador la entrada; si algo falla, no se mueve un solo céntimo.
- **Total transparencia:** Cualquiera puede comprobar en todo momento cuántas entradas quedan, el precio oficial y el historial de cada localidad sin depender de empresas privadas.

Diseño del Sistema

En este apartado definimos cómo funciona el contrato inteligente: las fases por las que pasa una entrada desde que se emite hasta que se valida en el recinto, los datos y funciones que utiliza el código, las reglas que nadie puede romper y los tres roles de usuarios que participan en el sistema.

2.1 Lógica de Negocio y Ciclo de Vida del Ticket

El contrato gestiona de forma automática todo el recorrido de una localidad a través de seis etapas:

Fase	Estado	Acción del Usuario	Seguridad Garantizada
Fase 0	Evento Creado	El organizador fija el aforo, el precio oficial y el recargo máximo de reventa.	Parámetros fijos que nadie puede alterar.
Fase 1	Compra Oficial	El usuario compra su entrada en taquilla pagando el precio oficial.	La entrada queda asignada a su cuenta digital. No vende si se agota el aforo.
Fase 2	Gestión del Dueño	El titular puede conservarla para asistir o ponerla en reventa si no puede ir.	Solo el dueño real puede ponerla en venta; precio dentro del límite.
Fase 3	Reventa Segura	Un segundo comprador paga el precio solicitado por el vendedor.	El dinero va al vendedor y la entrada al nuevo comprador al mismo tiempo.
Fase 4	Acceso en Puerta	El organizador escanea y valida la entrada en el acceso.	La entrada queda marcada como usada. Nadie puede reutilizarla.
Fase 5	Cobro de Taquilla	El organizador retira los fondos acumulados de la venta inicial.	Solo el organizador autorizado puede retirar ese dinero.

2.2 Contenido del Contrato: Datos, Funciones y Reglas de Operación

2.2.1 A. Información Almacenada (Datos de cada Entrada)

Para cada entrada emitida, el contrato guarda cuatro datos esenciales:

- **`owner`:** La dirección de la cuenta digital del dueño actual de la entrada.
- **`isUsed`:** Un indicador que marca si la entrada ya fue escaneada en el acceso (`true` o `false`).
- **`isForSale`:** Un indicador que señala si la entrada está publicada en el mercado de reventa.
- **`resalePrice`:** El precio que pide el dueño por ceder su entrada a otro usuario.

Además, el contrato guarda de forma fija los datos generales del evento: quién es el organizador (`organizer`), el precio original (`ticketPrice`), el porcentaje de recargo máximo permitido (`maxMarkupPercentage`, ej. 10 para un $+10\,\%$), el precio techo que nadie puede superar (`maxResalePrice`) y el aforo total (`totalTickets`).

2.2.2 B. Funciones del Contrato

El contrato cuenta con 7 funciones claras para operar con las entradas:

1. `buyTicket()` : Permite comprar una entrada oficial pagando el importe exacto.
2. `listTicketForResale(id, precio)` : Permite al dueño poner su entrada a la venta, comprobando que no supere el precio máximo.
3. `cancelResale(id)` : Permite al dueño retirar su oferta de reventa si cambia de opinión antes de que la compren.
4. `buyResaleTicket(id)` : Permite a otro usuario comprar una entrada de segunda mano; el dinero pasa al vendedor y la entrada al comprador en el mismo instante.
5. `validateAccess(id)` : Permite al organizador comprobar la entrada en puerta e invalidarla de por vida para que nadie pueda volver a entrar con ella.
6. `withdraw()` : Permite al organizador pasar la recaudación oficial de la taquilla a su cuenta personal.
7. `getTicket(id)` : Permite a cualquiera consultar públicamente el estado y datos de una entrada.

2.2.3 C. Reglas de Operación Inviolables

El contrato impone cuatro reglas obligatorias que nadie puede saltarse:

- **Respeto estricto del aforo:** Nadie puede comprar si ya se han vendido todas las localidades.
- **Prohibición de precios abusivos:** Si alguien intenta revender por encima del tope fijado, el contrato cancela la operación automáticamente.
- **Uso único de la entrada:** Una entrada escaneada queda inservible: nunca se puede volver a validar en puerta ni volver a vender.
- **Prohibición de auto-compra:** El dueño de una entrada no puede comprarse a sí mismo su propia reventa.

2.3 Actores del Sistema y Diagrama de Casos de Uso

En el sistema participan tres tipos de usuarios:

1. **Organizador del evento:** Crea el evento, controla las puertas de entrada al recinto y cobra la taquilla inicial.
2. **Comprador oficial (primer dueño):** Compra su entrada en taquilla y tiene derecho a revenderla a precio justo si no puede asistir.
3. **Comprador secundario:** Compra de segunda mano con la total seguridad de que la entrada es auténtica y de que el intercambio de dinero y titularidad se ejecuta en el mismo instante.

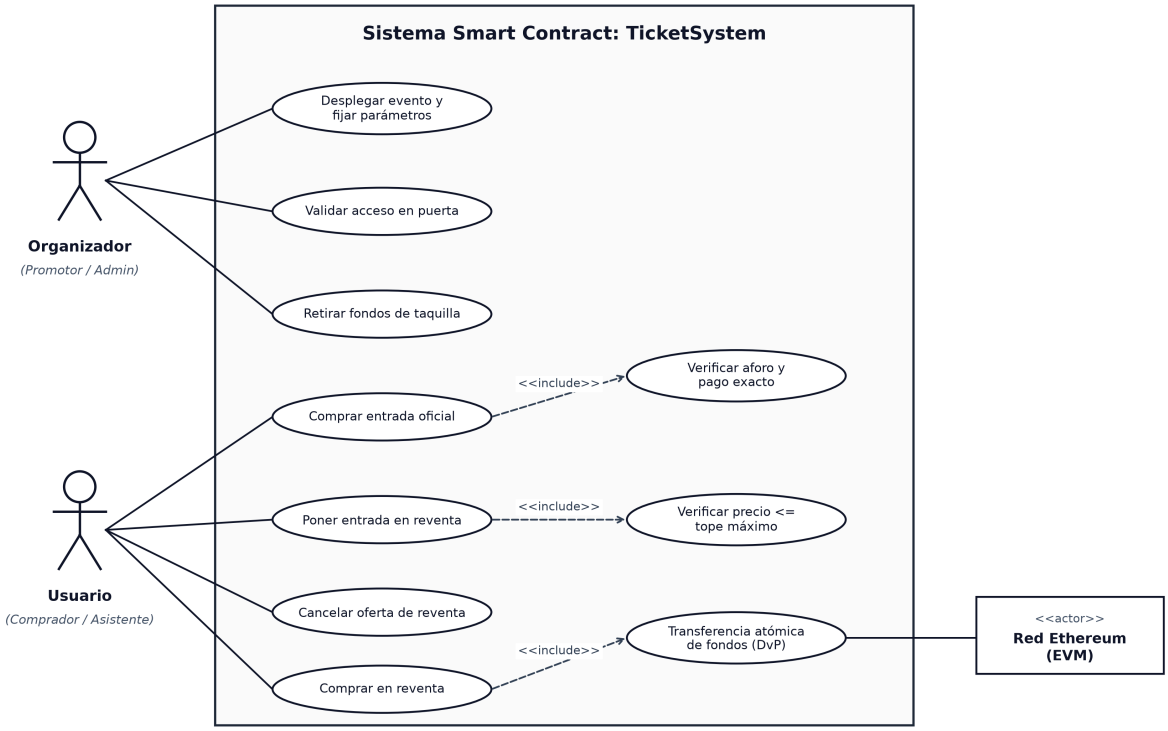


Figura 1: *Diagrama de Casos de Uso UML formal del sistema* `TicketSystem` *en Ethereum.*

Implementación y Optimización de Gas

El contrato inteligente se ha programado en Solidity `^0.8.20` buscando que cada operación sea lo más barata posible para los usuarios, aplicando cuatro mejoras clave:

1. **Avisos de error directos:** En lugar de guardar mensajes de texto largos para avisar cuando algo falla, el contrato utiliza identificadores numéricos directos. Esto hace que el contrato ocupe menos espacio y cueste mucho menos desplegarlo y usarlo.
2. **Agrupación de datos de cada entrada:** La información del dueño y el estado de la entrada se guardan juntos en un único espacio de memoria. Así, cuando una entrada cambia de manos, el contrato solo tiene que escribir en la base de datos una vez en vez de tres.
3. **Datos fijos que no cambian:** Los valores que se fijan al crear el evento (como el aforo total o el precio oficial) se graban directamente en el código base. Gracias a esto, el contrato no gasta comisión en buscarlos cada vez que alguien compra.
4. **Cobro seguro antes de transferir dinero:** Primero se actualiza el nuevo dueño en el sistema y después se envía el dinero al vendedor. De este modo, se asegura la operación sin necesidad de usar programas de seguridad externos que encarecerían la transacción.

3.1 Código del Smart Contract (`TicketSystemBasic.sol`)

A continuación se recoge el código completo del contrato inteligente implementado:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title TicketSystemBasic
 * @notice Sistema de emisión, venta y reventa controlada de entradas
 *         para eventos con límite máximo de precio de reventa y validación en acceso.
 * @dev Implementa optimización de gas mediante Custom Errors, Struct Packing e Immutable variables.
 */
contract TicketSystemBasic {

    // ============================================================================
    // CUSTOM ERRORS (Optimización de Gas - EIP-838)
    // ============================================================================
    error OnlyOrganizerAllowed();
    error OnlyTicketOwnerAllowed();
    error SoldOut();
    error IncorrectPayment(uint256 expected, uint256 provided);
    error TicketDoesNotExist(uint256 ticketId);
    error TicketNotForSale(uint256 ticketId);
    error TicketAlreadyUsed(uint256 ticketId);
    error CannotBuyOwnTicket();
    error PriceExceedsMaxLimit(uint256 proposedPrice, uint256 maxAllowed);
    error InvalidPrice();
    error TransferFailed();
    error NoFundsToWithdraw();

    // ============================================================================
    // ESTRUCTURA DE DATOS (Struct Packing: 22 bytes en el primer slot EVM)
    // ============================================================================
    struct Ticket {
        address owner;       // 20 bytes
        bool isUsed;         // 1 byte
        bool isForSale;      // 1 byte
```

```
        // Total Slot 1: 22 bytes <= 32 bytes (Ahorro de un SSTORE de 20.000 gas)
        uint256 resalePrice; // 32 bytes en Slot 2
    }

    // ========================================================================
    // VARIABLES DE ESTADO INMUTABLES Y DE ALMACENAMIENTO
    // ========================================================================
    address public immutable organizer;              // Dirección del organizador (PUSH32, 3 gas)
    uint256 public immutable ticketPrice;            // Precio original en Wei
    uint256 public immutable maxMarkupPercentage;    // Porcentaje máx. sobreprecio reventa (ej. 10 = +10%)
    uint256 public immutable maxResalePrice;         // Precio tope calculado en el despliegue
    uint256 public immutable totalTickets;           // Aforo total

    uint256 public ticketsSold;                      // Contador de entradas vendidas
    mapping(uint256 => Ticket) public tickets;       // ticketId => Ficha de la entrada

    // ========================================================================
    // EVENTOS
    // ========================================================================
    event TicketPurchased(uint256 indexed ticketId, address indexed buyer, uint256 price);
    event TicketListedForResale(uint256 indexed ticketId, address indexed seller, uint256 resalePrice);
    event ResaleCancelled(uint256 indexed ticketId, address indexed seller);
    event ResaleTicketSold(uint256 indexed ticketId, address indexed previousOwner, address indexed newOwner,
uint256 price);
    event TicketValidated(uint256 indexed ticketId, address indexed attendee);
    event FundsWithdrawn(address indexed organizer, uint256 amount);

    // ========================================================================
    // MODIFICADORES
    // ========================================================================
    modifier onlyOrganizer() {
        if (msg.sender != organizer) revert OnlyOrganizerAllowed();
        _;
    }

    modifier onlyTicketOwner(uint256 _ticketId) {
        if (_ticketId == 0 || _ticketId > ticketsSold) revert TicketDoesNotExist(_ticketId);
        if (tickets[_ticketId].owner != msg.sender) revert OnlyTicketOwnerAllowed();
        _;
    }

    // ========================================================================
    // CONSTRUCTOR
    // ========================================================================
    constructor(
        uint256 _totalTickets,
        uint256 _ticketPrice,
        uint256 _maxMarkupPercentage
    ) {
        if (_totalTickets == 0) revert InvalidPrice();
        if (_ticketPrice == 0) revert InvalidPrice();

        organizer = msg.sender;
        totalTickets = _totalTickets;
        ticketPrice = _ticketPrice;
        maxMarkupPercentage = _maxMarkupPercentage;
        maxResalePrice = _ticketPrice + ((_ticketPrice * _maxMarkupPercentage) / 100);
    }

    // ========================================================================
    // VENTA INICIAL (COMPRA PRIMARIA)
    // ========================================================================
    function buyTicket() external payable returns (uint256 ticketId) {
        if (ticketsSold >= totalTickets) revert SoldOut();
        if (msg.value != ticketPrice) revert IncorrectPayment(ticketPrice, msg.value);

        unchecked {
            ticketId = ++ticketsSold;
        }
```

```
        tickets[ticketId] = Ticket({
            owner: msg.sender,
            isUsed: false,
            isForSale: false,
            resalePrice: 0
        });

        emit TicketPurchased(ticketId, msg.sender, ticketPrice);
    }

    // ========================================================================
    // MERCADO SECUNDARIO / REVENTA CONTROLADA
    // ========================================================================
    function listTicketForResale(uint256 _ticketId, uint256 _price) external onlyTicketOwner(_ticketId) {
        Ticket storage ticket = tickets[_ticketId];

        if (ticket.isUsed) revert TicketAlreadyUsed(_ticketId);
        if (_price == 0) revert InvalidPrice();
        if (_price > maxResalePrice) revert PriceExceedsMaxLimit(_price, maxResalePrice);

        ticket.isForSale = true;
        ticket.resalePrice = _price;

        emit TicketListedForResale(_ticketId, msg.sender, _price);
    }

    function cancelResale(uint256 _ticketId) external onlyTicketOwner(_ticketId) {
        Ticket storage ticket = tickets[_ticketId];
        if (!ticket.isForSale) revert TicketNotForSale(_ticketId);

        ticket.isForSale = false;
        ticket.resalePrice = 0;

        emit ResaleCancelled(_ticketId, msg.sender);
    }

    function buyResaleTicket(uint256 _ticketId) external payable {
        if (_ticketId == 0 || _ticketId > ticketsSold) revert TicketDoesNotExist(_ticketId);

        Ticket storage ticket = tickets[_ticketId];

        if (!ticket.isForSale) revert TicketNotForSale(_ticketId);
        if (ticket.isUsed) revert TicketAlreadyUsed(_ticketId);
        if (msg.sender == ticket.owner) revert CannotBuyOwnTicket();
        if (msg.value != ticket.resalePrice) revert IncorrectPayment(ticket.resalePrice, msg.value);

        address previousOwner = ticket.owner;
        uint256 salePrice = ticket.resalePrice;

        // Efectos de estado antes de transferencias (Patrón CEI)
        ticket.owner = msg.sender;
        ticket.isForSale = false;
        ticket.resalePrice = 0;

        // Transferencia atómica directa de fondos al vendedor original
        (bool sent, ) = payable(previousOwner).call{value: salePrice}("");
        if (!sent) revert TransferFailed();

        emit ResaleTicketSold(_ticketId, previousOwner, msg.sender, salePrice);
    }

    // ========================================================================
    // CONTROL DE ACCESO EN PUERTA
    // ========================================================================
    function validateAccess(uint256 _ticketId) external onlyOrganizer {
        if (_ticketId == 0 || _ticketId > ticketsSold) revert TicketDoesNotExist(_ticketId);

        Ticket storage ticket = tickets[_ticketId];
```

```
        if (ticket.isUsed) revert TicketAlreadyUsed(_ticketId);

        ticket.isUsed = true;
        if (ticket.isForSale) {
            ticket.isForSale = false;
            ticket.resalePrice = 0;
        }

        emit TicketValidated(_ticketId, ticket.owner);
    }

    // ========================================================================
    // RETIRADA DE FONDOS DEL ORGANIZADOR
    // ========================================================================
    function withdraw() external onlyOrganizer {
        uint256 contractBalance = address(this).balance;
        if (contractBalance == 0) revert NoFundsToWithdraw();

        (bool success, ) = payable(organizer).call{value: contractBalance}("");
        if (!success) revert TransferFailed();

        emit FundsWithdrawn(organizer, contractBalance);
    }

    // ========================================================================
    // CONSULTAS / VISTAS PÚBLICAS
    // ========================================================================
    function getTicket(uint256 _ticketId) external view returns (
        address owner,
        bool isUsed,
        bool isForSale,
        uint256 resalePrice
    ) {
        if (_ticketId == 0 || _ticketId > ticketsSold) revert TicketDoesNotExist(_ticketId);
        Ticket storage t = tickets[_ticketId];
        return (t.owner, t.isUsed, t.isForSale, t.resalePrice);
    }

    function getAvailableInitialTickets() external view returns (uint256) {
        return totalTickets - ticketsSold;
    }
}
```

Pruebas y Validación en Remix IDE

En esta sección se comprueba el funcionamiento del contrato simulando el ciclo completo del evento: la venta inicial de entradas, el intento de reventa con sobreprecio, la compra de segunda mano, el control de acceso en la entrada y la retirada final del dinero.

4.1 Entorno de Pruebas y Cuentas Asignadas

Las pruebas fueron ejecutadas en **Remix IDE** utilizando el entorno virtual `Remix VM (Shanghai)` con tres cuentas de prueba:

- **Cuenta 1 (`0x5B3...eddC4`):** Rol de **Organizador** del evento deportivo / promotor.
- **Cuenta 2 (`0xAb8...35cb2`):** Rol de **Comprador 1** (asistente inicial).
- **Cuenta 3 (`0x4B2...C02db`):** Rol de **Comprador 2** (usuario del mercado de reventa).

4.2 Batería de Pruebas Ejecutadas Paso a Paso

4.2.1 Paso 1: Despliegue del Contrato y Fijación de Parámetros

El organizador crea el evento en la blockchain y define el aforo, el precio de salida y el porcentaje máximo de reventa permitido.

- **Cuenta:** `0x5B3...` (Organizador).
- **Parámetros:** `_totalTickets = 5`, `_ticketPrice = 1000000000000000000` (1 ETH), `_maxMarkupPercentage = 10` (+10 \ %).
- **Resultado:** Contrato desplegado con éxito en la dirección asignada. La función pública `maxResalePrice` computa de forma inmutable el valor `1100000000000000000` (1.1 ETH).

4.2.2 Paso 2: Venta Primaria Oficial

Un usuario compra la primera entrada en la taquilla oficial pagando el precio fijado.

- **Cuenta:** `0xAb8...` (Comprador 1).
- **Transacción:** Envío de `1 Ether` mediante la función `buyTicket()`.
- **Resultado:** Transacción minada con éxito (`TicketPurchased`). El contador `ticketsSold` pasa a 1 y el contrato custodia 1 ETH.

4.2.3 Paso 3: Demostración de Bloqueo ante Intento de Especulación Abusiva

El comprador intenta poner su entrada a la venta pidiendo el doble de su valor para comprobar si el sistema frena el sobreprecio.

- **Cuenta:** `0xAb8...` (Comprador 1, dueño del ticket 1).
- **Transacción:** Intento de listado mediante `listForResale(1, 2000000000000000000)` solicitando 2 ETH (sobreprecio del +100 \ % frente al +10 \ % permitido).
- **Resultado: Reversión automática de la transacción** por la EVM con el error personalizado:

```
Error provided by the contract: PriceExceedsMaxLimit(2000000000000000000, 1100000000000000000)
```

Demuestra que el contrato impide la especulación abusiva de forma autónoma.

4.2.4 Paso 4: Puesta en Reventa Legal

El comprador pone de nuevo su entrada en venta respetando el límite máximo de precio permitido por el organizador.

- **Cuenta:** `0xAb8...` (Comprador 1).
- **Transacción:** Llamada a `listForResale(1, 1050000000000000000)` (1.05 ETH, respetando el límite legal).
- **Resultado:** Éxito. El ticket queda marcado con `isForSale = true` y precio fijado en 1.05 ETH.

4.2.5 Paso 5: Compra de Segunda Mano con Liquidación Atómica

Un segundo usuario adquiere la entrada en reventa, pagando directamente al vendedor y recibiendo la titularidad al mismo tiempo.

- **Cuenta:** `0x4B2...` (Comprador 2).
- **Transacción:** Llamada a `buyResaleTicket(1)` enviando `1.05 Ether` (`Value = 1050000000000000000 Wei`).
- **Resultado:** Ejecución exitosa de liquidación atómica:
  1. Los 1.05 ETH son transferidos íntegramente a la cartera de la Cuenta 2 (`0xAb8...`).
  2. La propiedad del ticket 1 pasa inmediatamente a la Cuenta 3 (`0x4B2...`).
  3. `isForSale` pasa a `false`.

4.2.6 Paso 6: Control de Acceso en Puerta y Detección de Duplicados

Se comprueba la entrada en el acceso al recinto y se simula un segundo intento de entrada con la misma localidad para verificar que queda inutilizada.

- **Cuenta:** `0x5B3...` (Organizador).
- **Transacción 1:** Llamada a `validateAccess(1)` con `Value = 0`.
  - **Resultado:** Acceso autorizado con éxito (`TicketValidated`). El ticket pasa a `isUsed = true`.
- **Transacción 2 (Simulación de fraude por copia o intento de doble uso):** Segundo intento con `validateAccess(1)`.
  - **Resultado: Reversión inmediata** con el error `TicketAlreadyUsed(1)`. Queda demostrado que ninguna entrada puede ser utilizada más de una vez.

4.2.7 Paso 7: Retirada de Fondos del Organizador

Una vez finalizadas las ventas oficiales, el organizador transfiere la recaudación inicial de la taquilla a su cuenta personal.

- **Cuenta:** `0x5B3...` (Organizador).
- **Transacción:** Llamada a `withdraw()`.
- **Resultado:** El balance de 1 ETH acumulado por la venta primaria es transferido a la cartera del organizador, dejando el balance del contrato en 0.